

Obliczanie pierwiastków zespolonych wielomianu o współczynnikach rzeczywistych metodą Bairstowa

Adam Kwiatkowski

Numer indeksu: 167957

1 Zastosowanie

Procedura `bairstow_method` oblicza pierwiastki zespolone wielomianu (1) o współczynnikach rzeczywistych w zmiennopozycyjnej arytmetyce przedziałowej

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 \quad (1)$$

2 Opis metody

Do znalezienia pierwiastków zespolonych wielomianu (1) stosuje się metodę Bairstowa, w której unika się rachunków na liczbach zespolonych. Metoda ta opiera się na twierdzeniu, że pierwiastki wielomianu

$$x^2 - rx - q \quad (2)$$

gdzie r i q oznaczają współczynniki rzeczywiste, są pierwiastkami wielomianu (1), gdy wielomian (1) dzieli się bez reszty przez wielomian (2). W ogólności wykonując dzielenie wielomianu (1) przez wielomian (2) mamy

$$p(x) = p_1(x)(x^2 - rx - q) + Ax + B \quad (3)$$

przy czym stopień wielomianu $p_1(x)$ jest nie większy niż $n - 2$, a wyrażenie $Ax + B$ jest resztą z dzielenia. Współczynniki A i B zależą od r i q :

$$A = A(r, q), \quad B = B(r, q)$$

Reszta z dzielenia będzie równa 0, jeżeli

$$A(r, q) = 0, \quad B(r, q) = 0$$

Powyższe równania są układem dwu równań nieliniowych o dwu niewiadomych r i q . Do ich rozwiązania możemy zastosować proces iteracyjny Newtona:

$$\begin{bmatrix} r_{j+1} \\ q_{j+1} \end{bmatrix} = \begin{bmatrix} r_j \\ q_j \end{bmatrix} - \begin{bmatrix} \frac{\partial A}{\partial r} & \frac{\partial A}{\partial q} \\ \frac{\partial B}{\partial r} & \frac{\partial B}{\partial q} \end{bmatrix}_{r=r_j, q=q_j}^{-1} \begin{bmatrix} A(r_j, q_j) \\ B(r_j, q_j) \end{bmatrix},$$

który można zapisać w postaci układu równań liniowych:

$$\begin{bmatrix} \frac{\partial A}{\partial r} & \frac{\partial A}{\partial q} \\ \frac{\partial B}{\partial r} & \frac{\partial B}{\partial q} \end{bmatrix}_{r=r_j, q=q_j} \begin{bmatrix} r_{j+1} \\ q_{j+1} \end{bmatrix} = \begin{bmatrix} \frac{\partial A}{\partial r} & \frac{\partial A}{\partial q} \\ \frac{\partial B}{\partial r} & \frac{\partial B}{\partial q} \end{bmatrix}_{r=r_j, q=q_j} \begin{bmatrix} r_j \\ q_j \end{bmatrix} - \begin{bmatrix} A(r_j, q_j) \\ B(r_j, q_j) \end{bmatrix}. \quad (4)$$

Wartości $A(r, q)$ i $B(r, q)$ można otrzymać za pomocą następujących wzorów rekurencyjnych:

$$\begin{aligned} b_0 &= a_n, & b_1 &= b_0 r + a_{n-1}, \\ b_i &= b_{i-2} q + b_{i-1} r + a_{n-i}, & i &= 2, 3, \dots, n-2, \\ A(r, q) &= b_{n-3} q + b_{n-2} r + a_1, & B(r, q) &= b_{n-2} q + a_0. \end{aligned}$$

Pochodne cząstkowe występujące w równaniach (4) wyznacza się na podstawie wzorów

$$\begin{aligned} \frac{\partial A(r, q)}{\partial r} &= r \bar{A}(r, q) + \bar{B}(r, q), & \frac{\partial A(r, q)}{\partial q} &= \bar{A}(r, q), \\ \frac{\partial B(r, q)}{\partial r} &= q \bar{A}(r, q), & \frac{\partial B(r, q)}{\partial q} &= \bar{B}(r, q), \end{aligned}$$

gdzie wartości $\bar{A}(r, q)$ i $\bar{B}(r, q)$ mogą być obliczone na podstawie podobnych wzorów rekurencyjnych, a mianowicie

$$\begin{aligned} c_0 &= b_0, & c_1 &= c_0 r + b_1, \\ c_i &= c_{i-2} q + c_{i-1} r + b_i, & i &= 2, 3, \dots, n-4, \\ \bar{A}(r, q) &= c_{n-5} q + c_{n-4} r + b_{n-3}, & \bar{B}(r, q) &= c_{n-4} q + b_{n-2}. \end{aligned}$$

3 Wywołanie funkcji

```
vector<T> bairstow_method(
    degree,
    coefficients,
    max_iterations,
    relative_error,
    double zerodet,
    st,
    it,
    input_type
);
```

4 Dane

- **degree** – stopień wielomianu.
- **coefficients** – tablica zawierająca współczynniki wielomianu (element `coefficients[i]` powinien zawierać współczynnik przy x^i , gdzie $i = 0, 1, \dots, n$).

- `max_iterations` – liczba iteracji.
- `relative_error` – błąd względny wyznaczania r i q .
- `zerodet` – zero maszynowe
- `input_type` – typ arytmetyki

5 Wyniki

- `vector<complex_interval<T>` – tablica dynamiczna zawierająca rozwiązanie, każdy element tablicy zawiera pierwiastek zespolony wielomianu.

6 Inne parametry

- `st` – zmienna która przechowuje jedną z następujących wartości:
 - `st = 1`, jeżeli `degree < 1` lub `max_iterations < 1` lub `relative_error <= 0` lub `zerodet <= 0`.
 - `st = 2`, jeżeli wyznacznik macierzy z równania (4) jest mniejszy niż `zerodet`
 - `st = 3`, jeżeli przekroczona została maksymalna liczba iteracji
- `it` – zmienna, która przechowuje ilość iteracji wykonanych przez funkcję

7 Typy parametrów

- `int`: `degree`, `max_iterations`, `st`, `it`, `input_type`
- `long double`: `relative_error`, `zerodet`
- `vector`: `coefficients`

8 Identyfikatory nielokalne

- `T` - dane typu `long double` lub `Interval<long double>`
- `complex_interval<T>` – nazwa typu strukturalnego reprezentująca dane zespolone o składowych typu `T`, zdefiniowanego następująco:

```
template<typename T>
struct complex_interval
{
    T real;
    T imag;
};
```

- `vector<T>` – nazwa typu reprezentującego tablicę dynamiczną (kontener standardowy) o elementach typu `T`.
- `vector<complex_interval<T>>` – nazwa typu reprezentującego tablicę dynamiczną, której elementami są struktury przedziałów zespolonych.
- `newton_output<T>` – nazwa typu strukturalnego reprezentująca dane zwracane przez funkcję `newton` typu `T`, zdefiniowana następująco:

```
template<typename T>
struct newton_output
{
    T r;
    T q;

    vector<T> coefficients;
};
```

9 Tekst funkcji

```
template<typename T>
vector<complex_interval<T>> bairstow_method(int degree, vector<T> coefficients,
    int max_iterations, long double relative_error, long double zerodet, int &st,
    int &it, int input_type)
{
    vector<complex_interval<T>> output;

    if (degree == 0)
        return output;

    if (degree == 1)
        return first_degree_roots<T>(coefficients);

    if (degree == 2)
        return second_degree_roots<T>(coefficients);

    newton_output result = newton<T>(coefficients, max_iterations, zerodet,
        relative_error, st, it, input_type);

    if(st != 0)
        return output;

    T a_coef;

    if constexpr (is_same_v<T, long double>)
        a_coef = 1.0;
    else if constexpr (is_same_v<T, Interval<long double>>)
        a_coef = IntRead<long double>("1.0");

    vector<complex_interval<T>> temp = second_degree_roots<T>
        ({-1 * result.q, -1 * result.r, a_coef});

    output.insert(output.end(), temp.begin(), temp.end());

    vector<complex_interval<T>> rest = bairstow_method<T>
        (static_cast<int>(result.coefficients.size() - 1), result.coefficients,
        max_iterations, relative_error, zerodet, st, it, input_type);

    output.insert(output.end(), rest.begin(), rest.end());

    return output;
}
```

10 Przykłady

a) Dane rzeczywiste dla arytmetyki zwykłej

- **Wielomian:** $P(x) = x^4 + 5x^3 + 10x^2 + 10x + 4$

Dane: 1,4 5,3 10,2 10,1 4,0, mit = 25, mincorr = 1e-16, zerodet = 1e-16

Wyniki:

$$\begin{aligned} - x_1 &= -1.00000000000000001E0 + 0.0000000000000000E-1i \\ - x_2 &= -2.0000000000000000E0 + 0.0000000000000000E-1i \\ - x_3 &= -1.0000000000000000E0 + 1.0000000000000000E0i \\ - x_4 &= -1.0000000000000000E0 - 1.0000000000000000E0i \end{aligned}$$

st = 0, it = 12

b) Dane rzeczywiste dla arytmetyki przedziałowej

- **Wielomian:** $P(x) = x^5 - 15x^4 + 85x^3 - 225x^2 + 274x - 120$

Dane: 1,5 -15,4 85,3 -225,2 274,1 -120,0, mit = 25, mincorr = 1e-16, zerodet = 1e-16

Wyniki:

$$\begin{aligned} - x_1 &= [4.0000000000000000E0, 4.0000000000000001E0] \quad w: 4.34E-19 + \\ &\quad [0.0000000000000000E-1, 0.0000000000000000E-1]i \quad w: 0.00E-1 \\ - x_2 &= [1.0000000000000000E0, 1.0000000000000001E0] \quad w: 2.17E-19 + \\ &\quad [0.0000000000000000E-1, 0.0000000000000000E-1]i \quad w: 0.00E-1 \\ - x_3 &= [2.9999999999999999E0, 3.0000000000000001E0] \quad w: 6.51E-19 + \\ &\quad [0.0000000000000000E-1, 0.0000000000000000E-1]i \quad w: 0.00E-1 \\ - x_4 &= [1.9999999999999999E0, 2.0000000000000000E0] \quad w: 5.42E-19 + \\ &\quad [0.0000000000000000E-1, 0.0000000000000000E-1]i \quad w: 0.00E-1 \\ - x_5 &= [5.0000000000000000E0, 5.0000000000000000E0] \quad w: 0.00E-1 + \\ &\quad [0.0000000000000000E-1, 0.0000000000000000E-1]i \quad w: 0.00E-1 \end{aligned}$$

st = 0, it = 9

c) Dane przedziałowe dla arytmetyki przedziałowej

- **Wielomian:** $P(x) = [0.99, 1.01]x^3 + [-2.01, -1.99]x^2 + [-1.01, -0.99]x + [1.99, 2.01]$
Dane: $[0.99, 1.01], 3 \quad [-2.01, -1.99], 2 \quad [-1.01, -0.99], 1 \quad [1.99, 2.01], 0$, mit = 25, mincorr = 1e-16, zerodet = 1e-16

Wyniki:

- $x_1 = [9.9904232734377764\text{E-}1, 9.9904232734377765\text{E-}1] \quad \text{w: } 1.63\text{E-}19 + [0.0000000000000000\text{E-}1, 0.0000000000000000\text{E-}1]\text{i} \quad \text{w: } 0.00\text{E-}1$
- $x_2 = [-9.9999982252513858\text{E-}1, -9.9999982252513857\text{E-}1] \quad \text{w: } 1.63\text{E-}19 + [0.0000000000000000\text{E-}1, 0.0000000000000000\text{E-}1]\text{i} \quad \text{w: } 0.00\text{E-}1$
- $x_3 = [1.9712355645837102\text{E}0, 2.0312798688213885\text{E}0] \quad \text{w: } 6.00\text{E-}2 + [0.0000000000000000\text{E-}1, 0.0000000000000000\text{E-}1]\text{i} \quad \text{w: } 0.00\text{E-}1$

st = 0, it = 4

d) Przypadek nieobsługiwany

- **Wielomian:** $P(x) = x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1$
Dane: 1,6 -6,5 15,4 -20,3 15,2 -6,1 1,0, mit = 25, mincorr = 1e-16, zerodet = 1e-16

Wyniki:

- st = 2